Prompt: Here is my report help me develop slides to present this topic
Reason: Began a chat to help developed key point of a presentation

# TimeSlice F1 API

Abstract
Prompt: Provide a brief summary (approx. 100-150 words) of the project's purpose, the solution's core architecture, the primary technology stack used, and the overall outcome.

1. Introduction
   The Time Slice API is designed to provide analytical insights extracted from Formula 1 (F1) lap data. Its primary goal is to bridge the gap between raw data APIs, such as OpenF1, and actionable analytical insights. Users can import data and run a variety of functions to gain insights from a dataset of their own creation. The purpose of this technical report is to outline the key design and architectural decisions made for the Time Slice API. It will provide a comprehensive justification for the chosen technology stack and development approach, demonstrating how generative AI can be successfully used to assist in the creation of such projects.

2. Key Design and Architectural Choices
   The system utilises a Monolithic N-Tier architecture, containerised and deployed as a single service on Render. This approach minimises the need for networking, avoiding associated latency issues. This containerised setup also optimises the configuration for Render, allowing for its effective use. The application architecture is structured as a vertical stack with three tiers: the API Layer, managing internet communication, the Business Logic Layer, which contains the core application rules and operations, and the Database Layer, which handles the database. This N-tiered approach ensures that changes to project requirements only necessitate modifications in the affected layer, rather than the entire design. This also enables testing components individually and more efficiently with a target test for each individual function.

This architecture is realised through a coordinated array of software components, beginning with the Entry Point (main.py), which serves as the application bootstrap. This initialises the FastAPI instance, which integrates the routing infrastructure, with the Routing Modules mapping the routing of the URL paths to the various Python functions. In order to maintain the integrity of the system, the input data is validated against the Pydantic Schemas, which check the JSON data integrity before entering the

system's logic. After the integrity check, the input data is passed to the Business Logic Layer, which consists of the services, implement the rules of the application's logic, as well as the database queries, which are passed to the database layer, where the data blueprint is defined using SQLAlchemy, along with the Database Manager, which connects to the Azure SQL Server.

Prompt: Discuss any critical design patterns or principles applied (e.g., MVC, Repository Pattern, RESTful API design).

3. Technology Stack

   3.1. Programming Language

   Python 3.13 was selected as the primary language for this API due to its large number of high-level frameworks, such as FastAPI and Pydantic. Other languages like C# and Java offer fast execution. However, in this basic iteration, we rely on external data sources (such as OpenF1), which have high response times. As the primary bottleneck is network latency from third-party APIs, the marginal performance gains of a lower-level language are outweighed by Python's superior development tooling. This choice also factors in prior technical experience gained during a second-year group project developing a Python web application, meaning that prior knowledge could be utilised in creating the API.

3.2. Application Frameworks and Libraries

Selecting Python was primarily driven by the availability of the FastAPI framework, which became the foundational and initial choice for the technology stack. Unlike alternatives such as Flask or Django, FastAPI natively supports asynchronous programming, which allows the Render service to host the web application more efficiently by handling concurrent requests without blocking. A key benefit during development is FastAPI's integration of OpenAPI (Swagger UI). This functionality is essential for rapid testing of developed functionality, as it generates request URLs and provides a useful UI for testing endpoints.

To ensure system integrity, Pydantic is used to enforce strict schemas across all endpoints. This guarantees that all requests return the appropriate status codes and that the API functions correctly by rejecting malformed data before it reaches the internal logic. Complementing this is SQLAlchemy, which was chosen for its ability to decouple application logic from raw SQL. Allowing queries to be written in Python inherently increases security against SQL Injection attacks. Furthermore, SQLAlchemy is portable, enabling seamless development as the project transitions from a simple local database in early iterations to a production Azure database with only minor configuration changes. Both libraries (Pydantic and SQLAlchemy) are industry standards, are exceptionally well-documented, and are ideally suited for a project of this professional scope.

### 3.3. Database Systems

When first developing, a local SQLite database was used, which served as an efficient way to store the initial lap summary data while building out endpoint functionality. This process was facilitated by SQLAlchemy, which allowed the application to interact with the database. The initial setup transitioned into a Relational SQL model to provide a more robust and scalable foundation. This selection was primarily driven by the vital benefits of ACID compliance (Atomicity, Consistency, Isolation, and Durability). Furthermore, the SQL model allows users to be linked to the lap summaries they have curated for analysis, which prepares the API for multiple users and hosting.

In the final iteration of the project, the database is hosted on Azure PostgreSQL. Cloud storage allows concurrent use of the database by clients and enables the API to be hosted remotely. Using a large provider such as Azure also removes security and infrastructure concerns of hosting using other methods. The pricing model, which charges primarily for interactions instead of uptime, makes it highly cost-effective at the project's current scale, with Azure's student credits making this a good option. This setup ensures that the data remains persistent in the cloud, independent of the API's remote hosting.

### 3.4. Deployment and CI/CD

To remotely host this API, I chose Render as it is highly compatible with my selected source control system of GitHub. Render generates a containerised instance of the repository and hosts it on a public web server that is accessible to anyone with access to the internet. A key advantage of this platform is that Render automatically updates with any pushes to the selected hosting branch, which means development remains simple and efficient. The scalability is also a core feature, with Render offering basic plans for current hosting requirements, but upgrade plans can be purchased if this application needs to scale in terms of user traffic or project size.

## 4. Reflection on Development

### 4.1. Testing Approach

Testing was undertaken in a range of forms throughout the development process, with the initial prototype being built out and checked using simple user testing facilitated by Swagger UI. However, this was not completely suitable for testing older features as the project grew. To ensure components stayed reliable through development, Pytest was set up to test module functionality. This was achieved by simulating a client instead of requesting actual web addresses, as hosting had not yet started. Pytest initially functioned through basic unit testing of endpoints, but once the full project was created, a further testing set was developed. Testing was conducted to confirm that all endpoints and hosting capabilities returned the expected status codes. A critical part of this was implementing edge testing to protect against invalid data, including the identification of potential edge cases and ensuring comprehensive coverage of all

error codes. Generative AI was utilised to assist in this role (as referenced in the GenAI declaration).

### 4.2. Challenges and Solutions

When developing, the client setup for Pytest was one of the hardest parts of the project, as it proved difficult to configure the client in an appropriate way to test functionality. This process was assisted by the debugging of Generative AI, which was used to help debug the configuration file and find workarounds for my exact API stack (as referenced in the GenAI declaration).

Another part of development that proved challenging was making the inputs user-friendly. OpenF1 requires a Session ID, which, like most primary keys, is inaccessible to the average user. By building on some of OpenF1's other endpoints, I was able to make this more user-friendly by converting the inputs of Year, Location, and Race Type. This did prove challenging to integrate into the base database and link between the user-friendly input, converting to a Session ID, collecting the data, and storing it; however, this was overcome by breaking down the steps and reforming the 'lap_summary' database structure to accommodate the conversion module.

### 4.3. Lessons Learned

Making an integrated, remotely hosted API taught me a lot, especially regarding database hosting through Azure and remote web hosting through Render. The use of both these systems required connection strings to be passed between them; this is a key process that must be kept secure to maintain the system. The use of .env files and storing these as Environment Variables was new to me, and I learned a lot about how to keep these credentials secure. This also required research to ensure Render was a secure and trusted platform to handle secret information, which was another important consideration.

This project also taught me a lot about securing and planning this type of development. I realised that ensuring tests are laid out sooner, and understanding how these tests will function reduces the overhead of initial user testing. This directly feeds into the importance of setting up the environment and sorting files correctly. I learned that a structured approach keeps files, such as the routers, relatively simple, with functionality spread across service and schema files. This modularity ensures the long-term maintainability and understanding of the overall system.

## 5. Limitations and Future Development

The project has various areas which would limit its expansion, but most have simple fixes. One such area is that the User System lacks the ability to change passwords, has no two-factor authentication, and does not enforce specific password requirements. These are all security features that could be resolved by building out a more comprehensive User system. While Render offers the facility for multiple users, if the scale becomes too large, a more premium subscription would likely be needed to handle the traffic.

Furthermore, the OpenF1 API data is a massive limiting factor. Retrieval times are currently very high, and there is significant rate limiting. Additionally, no data can be pulled during a live session. To mitigate this in the future, I would introduce a separate, dedicated database containing all historical data. This would significantly reduce latency on requests and remove downtime caused by external API constraints.
To improve the overall project, I would focus on building a Front-End Interface and an advanced User System to make the endpoints more accessible. This would allow the application to visualise output data graphically, making it more than just a theoretical JSON response. Additionally, introducing more analytical tools and endpoints would scale the project and improve its utility. Creating custom models for factors such as tyre degradation and race strategy would be an interesting next step. Building these features off the previously mentioned historical data would allow for more insightful and reliable race analysis features.

6. Genrtive AI
   Prompt: Declaration: State clearly whether Generative AI (GenAI) tools (e.g., GitHub Copilot, ChatGPT, Bard) were used in the development of this project.
   If GenAI was NOT used: Briefly state that GenAI was not used and move to the conclusion.
   If GenAI WAS used:
   Prompt: Provide a thoughtful analysis of its usage. Specify where (e.g., code generation, documentation, debugging) and how much (e.g., percentage of code base, specific complex functions).
   Prompt: Reflect on the benefits and drawbacks of using GenAI in this project.
   Prompt: Discuss the ethical and integrity considerations surrounding the code or ideas generated by AI. this is the report i produce form it make this into slide form with just key point detial the itnro to the proejct the teck then ai was used

I've created a professional presentation covering your TimeSlice F1 API project. The 15 slides include:

**Project Introduction** (slides 1-3): Title, overview, and architecture design highlighting the N-tier approach

**Technology Stack** (slides 4-9): Comprehensive coverage of Python/FastAPI selection, data integrity with Pydantic/SQLAlchemy, database evolution from SQLite to Azure PostgreSQL, and Render deployment

**Development Process** (slides 10-12): Testing methodology, challenges faced, and key lessons learned

**AI Usage** (slide 13): Clear statement on generative AI integration throughout development, particularly for Pytest configuration and debugging

**Future Work** (slide 14): Planned enhancements including user system improvements, historical database, and front-end visualization

The dark theme with blue accents creates a modern, technical aesthetic suitable for a software engineering presentation. Each slide focuses on one key concept with concise bullet points (4-5 maximum) for clarity during presentation delivery.

⁂

# let help produce it lets detial the sldie 1-3

Here's a concrete draft for slides 1–3 with just the key points you can paste into PowerPoint/Slides.

## Slide 1 – Title

**Title:** TimeSlice F1 API
**Subtitle:** Turning Raw F1 Lap Data into Actionable Insights

- Developer: Sam Hurst (ID: 201606431)
- Institution: University of Leeds
- Tech focus: Python, FastAPI, Azure, PostgreSQL
- Deployed as a publicly accessible REST API

## Slide 2 – Project Overview

**Project Purpose**

- Provides analytical insights from Formula 1 lap data
- Bridges gap between raw APIs (e.g. OpenF1) and usable analysis
- Lets users import data and run analysis on their own curated datasets

- Technical report focuses on design, architecture, and tech stack choices
- Demonstrates how **generative** AI can assist in building such an API

### Slide 3 – Core Architecture

**Monolithic N-Tier Design**

- Monolithic service, containerised and deployed on Render
- Vertical 3-tier architecture:
  - API Layer – FastAPI, routing, HTTP handling
  - Business Logic Layer – services, rules, analytical operations
  - Database Layer – SQLAlchemy models and DB manager (Azure PostgreSQL)
- Reduces network latency by keeping everything in one container
- Supports modular changes and targeted testing at the function/layer level

# Problem Context  talk about how it was a universty probejct i chose to higlight it as intresting in the world of ai and is very recent show how current work flow summries into bullpoint to tlak about

You can frame this as a short "Problem Context" section that links your uni project to current AI-powered dev workflows. Here's a slide-ready version.

### Slide X – Problem Context

**University Project in a Changing Landscape**

- Developed as an individual University of Leeds software engineering project, with full lifecycle: design, implementation, testing, and evaluation.[11][12]

- Chosen as a showcase because it combines traditional backend/API engineering with modern AI-assisted development practices, making it highly relevant to 2026 workflows.[13][14]
- Sits in a space where developers increasingly use AI tools for code generation, debugging, and documentation rather than purely manual workflows.[14][13]
- Demonstrates how a relatively small student project can still mirror industry practices: cloud hosting, CI-style deployment, and structured testing supported by AI tooling.[15][14]
- Provides a concrete, recent example of how human-led design decisions and **generative** AI support can be combined responsibly within an academic setting, aligned with emerging university guidance on ethical GenAI use.[16][17]

If you tell me where you want this in the deck (e.g., as Slide 3 or 4) I can help you link it cleanly from the intro.

⁂

# split into two slide first bing probelm contex keep bullet point short an dsweet but also have info to put in presenting notes and resosn for higlighting bring up that stuff

Here's a clean split into two slides, with short bullets plus suggested speaker notes.

### Slide 4 – Problem Context (1/2)

- Individual University of Leeds software engineering project
- Full lifecycle: design, build, test, evaluate
- Focus on a real F1 data use case
- Built to mirror "real" backend/API work

**Presenter notes:**
This was not just a small coding exercise but a full university project where I owned the entire lifecycle: requirements, architecture, implementation, and evaluation. I chose Formula 1 data because

it is time-critical, noisy, and very relevant to analytics, which makes it a good proxy for real-world data engineering problems. I also deliberately treated it like an industry backend: clear layers, an external data provider (OpenF1), a cloud database, and remote hosting.

### Slide 5 – Why Highlight This Project? (2/2)

- Very recent: reflects 2025–26 tech stack and practices
- Combines classic API engineering with GenAI support
- Uses cloud, CI-style deployment, and automated tests
- Shows how students now work with AI, not just code by hand

**Presenter notes:**
I chose to highlight this project because it is recent and uses the same tools teams are using today: FastAPI, cloud databases, containerised deployment, and pytest-based testing. What makes it interesting in the current AI landscape is that I used generative AI as a helper: to debug tricky Pytest client setup, to explore design options, and to refine documentation, rather than to "write my project for me". It captures what a modern workflow looks like for a student developer: human-led design and ownership, with AI as an assistant embedded in the process.

# Talk about highlevle stack and low level stack on spearetae slides highlevel is the hsoting wiht render azure data base and n-terir monthlic arture of code then low level is the code stack used to actualy make python tooling

You can present this as two clean slides: "High-Level Stack" and "Low-Level Stack". Here's slide-ready text plus what to say.

### Slide X – High-Level Stack (Architecture & Hosting)

**On the outside (where it runs):**

- Monolithic N-tier backend: API, business logic, database[21][22]
- Hosted as a single containerised service on Render (GitHub-connected auto-deploy)
- Production database hosted on Azure PostgreSQL in the cloud
- External data source: OpenF1 API for raw Formula 1 lap/session data

**Presenter notes:**
At the high level, TimeSlice is a single monolithic service, but internally it is still structured into tiers: the API layer, the business logic layer, and the database layer, following standard N-tier principles on Azure-style architectures. Render hosts the container built from the GitHub repo and redeploys automatically on push, which gives me a lightweight CI/CD feel without running my own pipeline. The API talks to an Azure-hosted PostgreSQL database for persistence, and it pulls raw race data from the OpenF1 API, so in practice the system coordinates three main components: Render (compute), Azure (state), and OpenF1 (data source).[22][21]

## Slide Y – Low-Level Stack (Code & Libraries)

**On the inside (how it's built):**

- Language: Python 3.13
- Web framework: FastAPI for async REST endpoints and OpenAPI docs[23]
- Data validation: Pydantic schemas for request/response models[23]
- ORM and DB access: SQLAlchemy for models and queries[24][23]
- Testing: Pytest with a test client simulating API calls
- Tooling: .env + environment variables for secrets, GitHub for version control

**Presenter notes:**
Zooming in, everything is implemented in Python. FastAPI provides the HTTP layer and generates the interactive OpenAPI/Swagger docs for quick manual testing. Pydantic models enforce strict schemas so malformed JSON never reaches the core logic, while SQLAlchemy lets me express queries as Python code rather than raw SQL, making it easier to switch from SQLite in early development to Azure PostgreSQL later. Pytest, together with FastAPI's test client, allowed me to validate endpoints and error handling before and after deployment. Secrets like connection strings are kept in .env files and passed to Render and Azure as environment variables rather than hard-coded in the repo.[24][23]

⁂

# Challenges and lesosn lerned sldie and creaetive use of gen ai slide please

Here are two slide-ready drafts plus speaker notes.

## Slide – Challenges & Lessons Learned

**Key challenges**

- Configuring a reliable Pytest client for FastAPI
- Making OpenF1 inputs user-friendly (year/location/race type → session ID)
- Designing a schema that links users to their curated lap summaries
- Managing secure connection strings across Render and Azure

**Lessons learned**

- Plan testing strategy and fixtures earlier, not at the end
- Keep routers thin and push logic into services and schemas
- Treat secrets as first-class: .env + environment variables only
- Structured architecture makes later changes much easier to reason about

**Presenter notes:**
One of the hardest parts was getting Pytest configured to talk to the FastAPI app correctly, with a client that behaved like a real user but without actually deploying the service. I also underestimated how tricky it would be to hide OpenF1's internal session IDs behind friendly inputs like year, location, and race type, while still keeping the database schema clean. Another big learning area was secrets management: I had to understand connection strings, .env files, and environment variables on both Render and Azure, and why you must never hard-code credentials. Overall, the project reinforced that planning tests, structure, and configuration up front saves a lot of pain later.

## Slide – Creative Use of Generative AI

**How GenAI was used**

- Debugging Pytest client and FastAPI test configuration[31][32]
- Suggesting edge cases and error-path tests for endpoints[33][31]
- Refactoring and clarifying complex service functions[34][32]
- Drafting and tightening documentation and report wording[35][31]

**Benefits vs drawbacks**

- Benefits: faster debugging, better test coverage, clearer explanations[32][34]
- Drawbacks: risk of over-trusting suggestions, occasional incorrect or insecure patterns[36][31]
- My rule: AI for ideas and drafts, but human review for all design and code decisions[31][35]

**Presenter notes:**
I used generative AI as a kind of "pair programmer", especially when wrestling with Pytest and FastAPI's test client setup. I would paste error messages and a simplified version of the code, then use the suggestions as a starting point rather than a final answer, which aligns with current best-practice guidance for researchers using AI for coding and debugging. It was also helpful for brainstorming edge cases and improving the wording of documentation and this report. On the flip side, the models sometimes suggested patterns that were either outdated or didn't quite fit my stack, so I had to stay critical and verify everything. The important part, especially in a university setting, is that all final design and code decisions remained my own, with AI used transparently and ethically as support rather than a substitute for my work.[35][36][31]

⁂


1. https://lsaglobal.com/technical-presentation-best-practices/
2. https://www.prezent.ai/blog/project-presentation
3. https://tosea.ai/blog/top-10-business-presentation-tips-2026
4. https://www.przntperfect.com/post/company-presentation
5. https://csmantech.org/wp-content/uploads/2025/12/2026-CS-MANTECH-Presentation-Guidelines.pdf
6. https://www.aippt.com/blog/technical-topics-for-presentation

7. https://slidemodel.com/project-presentation/

8. https://www.chatslide.ai/articles/industry-specific-presentation-design-trends-2026

9. https://libguides.hull.ac.uk/present/structure

10. https://www.salamandra.uk/news/best-practices-for-acing-your-presentation-design

11. https://pureadmin.qub.ac.uk/ws/portalfiles/portal/419912660/2022075749.pdf

12. https://www.comp.nus.edu.sg/~damithch/guide/context.html

13. https://www.baytechconsulting.com/blog/unlocking-ai-software-development-2026

14. https://devactivity.com/posts/apps-tools/the-ai-powered-dev-workflow-reshaping-software-engineering-in-2026/

15. https://knowledge.kitchen/content/courses/software-engineering/presentations/

16. https://www.kcl.ac.uk/about/strategy/learning-and-teaching/ai-guidance/student-guidance

17. https://intranet.birmingham.ac.uk/student/libraries/asc/student-guidance-gai.aspx

18. https://stackoverflow.com/questions/1570272/giving-presentation-on-software-project-to-non-programmers

19. https://home.cs.colorado.edu/~kena/classes/5828/s12/lectures/28-sepresentations.pdf

20. https://www.youtube.com/watch?v=1M_Mr6jdRdk

21. https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier

22. https://video2.skills-academy.com/en-us/azure/architecture/guide/architecture-styles/n-tier

23. https://blog.stackademic.com/using-fastapi-with-sqlalchemy-5cd370473fe5

24. https://github.com/fastapi/fastapi/discussions/9084

25. https://www.reddit.com/r/networking/comments/6rm1gd/high_and_low_level_network_designs/

26. https://www.geeksforgeeks.org/system-design/difference-between-high-level-design-and-low-level-design/

27. https://www.baeldung.com/cs/hld-lld

28. https://www.systemdesignhandbook.com/blog/low-level-design-vs-high-level-design/

29. https://www.youtube.com/watch?v=H703ErIrby8

30. https://getsdeready.com/high-level-vs-low-level-system-design-key-differences-and-use-cases/

31. https://www.turing.ac.uk/blog/using-generative-ai-write-code-guide-researchers

32. https://www.lockedinai.com/blog/generative-ai-in-debugging-best-practices-2025

33. https://stackoverflow.com/questions/73295786/how-can-i-test-for-exception-cases-in-fastapi-with-pytest

34. https://www.eejournal.com/article/using-generative-ai-for-refactoring-and-debugging-code-cut-debugging-time-in-half/

35. https://creativecomputing.gse.harvard.edu/genai-sdp.html

36. http://www.diva-portal.org/smash/get/diva2:1977937/FULLTEXT02.pdf

37. https://www.smartsheet.com/content/lessons-learned-template

38. https://www.linkedin.com/posts/paul-wishart_ai-generativeai-beeboistheonetruegod-activity-7373485266709299200-BGAH

39. https://www.projectmanagertemplate.com/post/lessons-learned-template-powerpoint-ultimate-guide

40. https://stackoverflow.com/questions/77111793/pytest-not-updating-to-new-api-between-test-scripts-fastapi